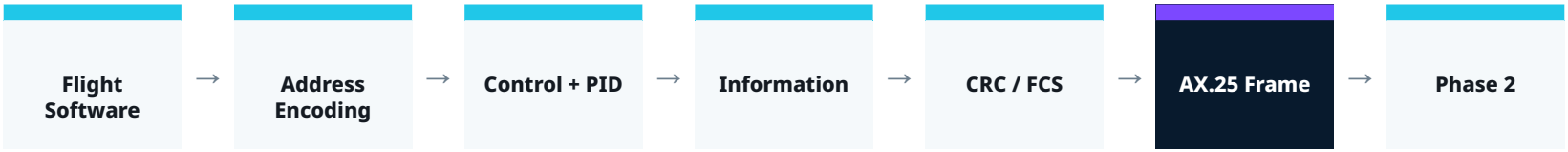
Phase 1 - AX.25 Frame Core

จาก Mission Data สู่ AX.25 UI Frame ระดับ Byte - เห็นเส้นทางข้อมูลก่อนลงรายละเอียด

1 Block Diagram - ข้อมูลไหลอย่างไร?

รับข้อมูลจาก Flight Software → สร้าง AX.25 frame → ส่งต่อให้ Phase 2 ทำ Bit / HDLC



INPUT - รับมาจากไหน?

Flight Software ส่งข้อมูลที่ต้องการสื่อสาร เช่น housekeeping telemetry. ตัวอย่าง prototype: `SEQ=1,BAT=7.60,TEMP=31.4,MODE=NOMINAL`

OUTPUT - ส่งต่อให้ใคร?

Phase 1 สร้าง **logical AX.25 frame** ที่ตรวจ CRC ได้แล้ว จากนั้น Phase 2 จะนำ frame body ไปแปลงเป็น bitstream, stuffing, HDLC และ NRZI.

ขอบเขตของ Phase 1: ทำงานระดับ **Byte / Frame**. ยังไม่ทำ Bit Stuffing, HDLC flag stream, NRZI หรือ RF modulation.

FLAG	DESTINATION	SOURCE	CONTROL	PID	INFORMATION	FCS	FLAG
7E	GROUND-0	ORBITL-1	03	F0	Telemetry	E2 E1	7E

GROUND-0 / ORBITL-1 เป็น test identifiers สำหรับ prototype เท่านั้น ยังไม่ใช่ final mission callsign.

2 แต่ละ Block ทำอะไร และใช้ตอนไหน?

อ่านจากซ้ายไปขวาตาม data flow ของหน้าแรก

1) Address Encoding

ทำอะไร: แปลง callsign + SSID เป็น AX.25 address field 7 bytes ต่อสถานี.

ใช้ตอนไหน: ทุกครั้งที่สร้าง frame เพื่อระบุว่า “ส่งจากใคร → ไปหาใคร”.

ตัวอย่าง:

GROUND-0 → `8E A4 9E AA 9C 88 60`

ORBITL-1 → `9E A4 84 92 A8 98 63`

2) Control Field

ทำอะไร: บอกชนิด/พฤติกรรมของ AX.25 frame.

ใช้ตอนไหน: เมื่อ receiver ต้องตีความว่า frame นี้เป็นรูปแบบใด.

Prototype: ใช้ `0x03` = UI (Unnumbered Information) frame เพื่อส่งข้อมูลโดยไม่ทำ connection-oriented sequencing ในชั้นนี้.

3) PID - Protocol ID

ทำอะไร: ระบุ protocol ของ payload ที่อยู่ใน Information Field.

ใช้ตอนไหน: receiver อ่านต่อจาก Control field เพื่อรู้ว่าจะตีความ payload ต่ออย่างไร.

Prototype: `0xF0` = no layer-3 protocol.

4) Information Field

ทำอะไร: เป็นพื้นที่เก็บข้อมูลจริงที่เราต้องการส่ง.

ใช้ตอนไหน: หลัง header fields ถูกกำหนดแล้ว ก่อนคำนวณ CRC.

ตอนนี้: ใช้ ASCII telemetry เพื่อทดสอบ software core. Phase 3 จะเปลี่ยนเป็น ORBITL-1 binary mission packet.

5) CRC-16/X-25 → FCS

ทำอะไร: ตรวจจับความผิดพลาดของข้อมูลใน frame body.

ใช้ตอนไหน: Sender คำนวณก่อนส่ง; receiver คำนวณซ้ำเมื่อรับ frame.

Known result: CRC numeric = `0xE1E2`; วางใน frame แบบ low-byte-first เป็น `E2 E1`.

6) Frame Assembly / Decoder

ทำอะไร: รวม Address + Control + PID + Information + FCS เป็น frame และ decoder แยกกลับ.

ใช้ตอนไหน: ก่อนส่งต่อ Phase 2 และเมื่อ Ground side ต้องตรวจ/อ่านข้อมูลกลับ.

ผล: logical frame ตัวอย่างยาว **57 bytes** รวม opening/closing flag.

เห็นภาพง่าย ๆ: Address = หน้าซอง, Control/PID = ป้ายบอกประเภท, Information = ของในซอง, FCS = ซีลตรวจว่าซองถูกแก้หรือเสียหายระหว่างทางหรือไม่.

CRC ไม่ได้แก้ error: มันทำหน้าที่ “ตรวจจับ” ว่าข้อมูลที่รับมาไม่ตรงกับที่ sender คำนวณไว้ แล้ว decoder จึง reject frame.

Phase 2 - AX.25 Bit / HDLC Layer

จาก logical frame body 55 bytes สู่ HDLC/NRZI bitstream - เห็นทั้งเส้นทางส่งและเส้นทางรับกลับ

1 Block Diagram - ข้อมูลไหลอย่างไร?

รับ frame body จาก Phase 1 → แปลงเป็นบิต → ป้องกัน flag collision → ใส่ HDLC boundary → เข้ารหัส NRZI → ส่งต่อ Physical/RF

Phase 1 AX.25 frame → **Frame Body** 55 bytes → **LSB-first** 440 bits → **Bit Stuffing** 441 bits → **HDLC Flags** 457 bits → **NRZI** 457 levels → **Physical / RF** Next integration

Receive / Decode Path: NRZI levels → NRZI decode → ตรวจ HDLC flags → de-stuff → bits-to-bytes → ได้ frame body 55 bytes เดิมกลับมา

INPUT - รับมาจากไหน?

รับ **AX.25 frame body 55 bytes** จาก Phase 1 โดยตัด logical opening/closing flag ระดับ byte ออกก่อน เพื่อไม่ให้ flag ถูกนำไป bit stuffing.

OUTPUT - ส่งต่อให้ใคร?

ได้ **NRZI level stream 457 levels** ที่พร้อมส่งต่อให้ modem / physical / RF chain. ตอนนี้ยังเป็น software-level 0/1 ไม่ใช่คลื่น RF จริง.

ขอบเขตของ Phase 2: ทำงานระดับ **Bit / HDLC / Line Coding**. ยังไม่ทำ modulation, symbol shaping, SDR hardware, Doppler หรือ RF link testing.

FRAME BODY	LSB-FIRST	STUFFED	HDLC + 2 FLAGS	NRZI
55 bytes	440 bits	441 bits	457 bits	457 levels

2 แต่ละ Block ทำอะไร และใช้ตอนไหน?

อ่านตาม data flow จาก Frame Body ไปถึง NRZI และย้อนกลับที่ receiver

1) Frame Body - 55 bytes

ทำอะไร: เป็นข้อมูล AX.25 จริงที่ต้องส่ง ได้แก่ address, control, PID, information และ FCS.

ใช้ตอนไหน: เป็น input ของ Phase 2 หลังถอด logical flags ของ Phase 1 ออก.

`57-byte frame → 55-byte body`

2) LSB-first Serialization

ทำอะไร: แปลงทุก byte เป็น 8 bits โดยส่ง bit ตำแหน่งต่ำสุดก่อน.

ใช้ตอนไหน: ก่อนทำ HDLC bit processing เพราะลำดับ bit บนสายส่งต้องชัดเจน.

ตัวอย่าง: `0x8E = 10001110 → 01110001`

3) Bit Stuffing

ทำอะไร: หลังเจอ **1 ติดต่อกัน 5 ตัว** ให้ insert 0 หนึ่งตัว.

ใช้ตอนไหน: ใช้เฉพาะข้อมูลภายใน frame เพื่อป้องกัน pattern ภายในไปเหมือน HDLC flag.

`440 bits → 441 bits` ใน test vector นี้เพิ่มจริง 1 bit.

4) HDLC Flags

ทำอะไร: ใส่ opening และ closing flag `01111110` อย่างละ 8 bits.

ใช้ตอนไหน: หลัง stuffing เสร็จ เพื่อให้ receiver รู้ขอบเขต frame.

`8 + 441 + 8 = 457 bits`

5) NRZI Encode / Decode

ทำอะไร: แปลง bit เป็นการเปลี่ยน/คงระดับสัญญาณ.

ใช้ตอนไหน: หลังประกอบ HDLC stream และก่อน physical/RF layer.

Convention: bit 0 = transition, bit 1 = no transition.

6) Reverse / Receive Path

ทำอะไร: ทำย้อนกลับ NRZI → HDLC → de-stuff → bits-to-bytes.

ใช้ตอนไหน: เมื่อ receiver ต้องกู้ frame body เดิมจาก bitstream ที่รับมา.

`Recovered body = 55 bytes` และต้องเท่าต้นฉบับ.

ทำไมลำดับต้องเป็นแบบนี้? ต้อง serialize ก่อนจึงจะตรวจ run ของบิตได้; stuffing ต้องเกิดก่อนใส่ flag เพราะ flag ห้ามถูก stuffed; NRZI ต้องทำหลังได้ HDLC bitstream ที่สมบูรณ์แล้ว.

Bytes → **LSB-first** → **Stuff** → **Add Flags** → **NRZI**

สิ่งที่ยังไม่ควรสรุป: PASS ถึง NRZI ไม่ได้แปลว่า RF link พร้อมใช้งาน. Modulation, symbol rate, filtering, SDR integration และ channel effects ยังอยู่นอก Phase 2.